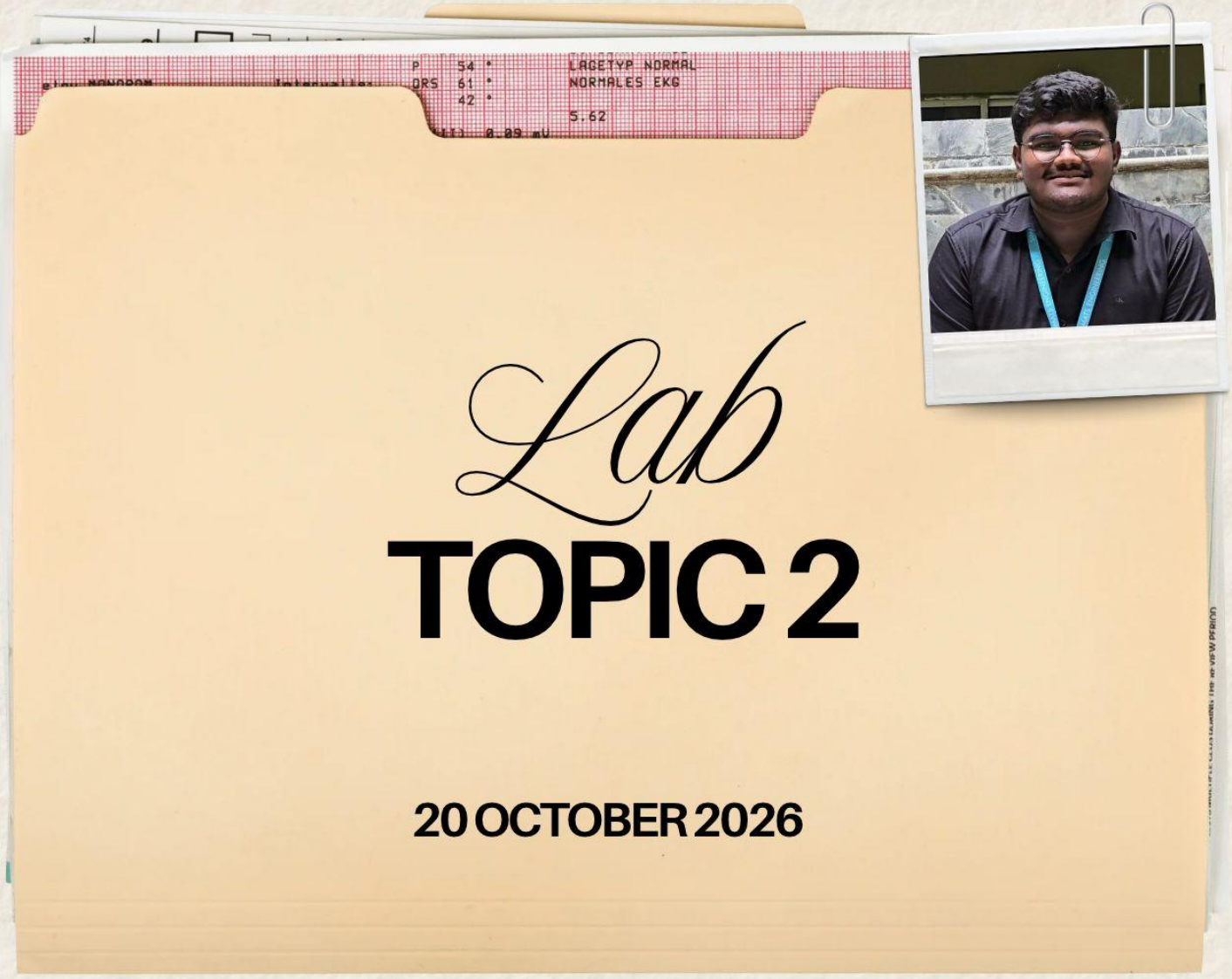
CSA0617 DESIGN AND ANALYSIS OF ALGORITHMS



Lab
TOPIC 2

20 OCTOBER 2026

192424401 JERRY DAVID R

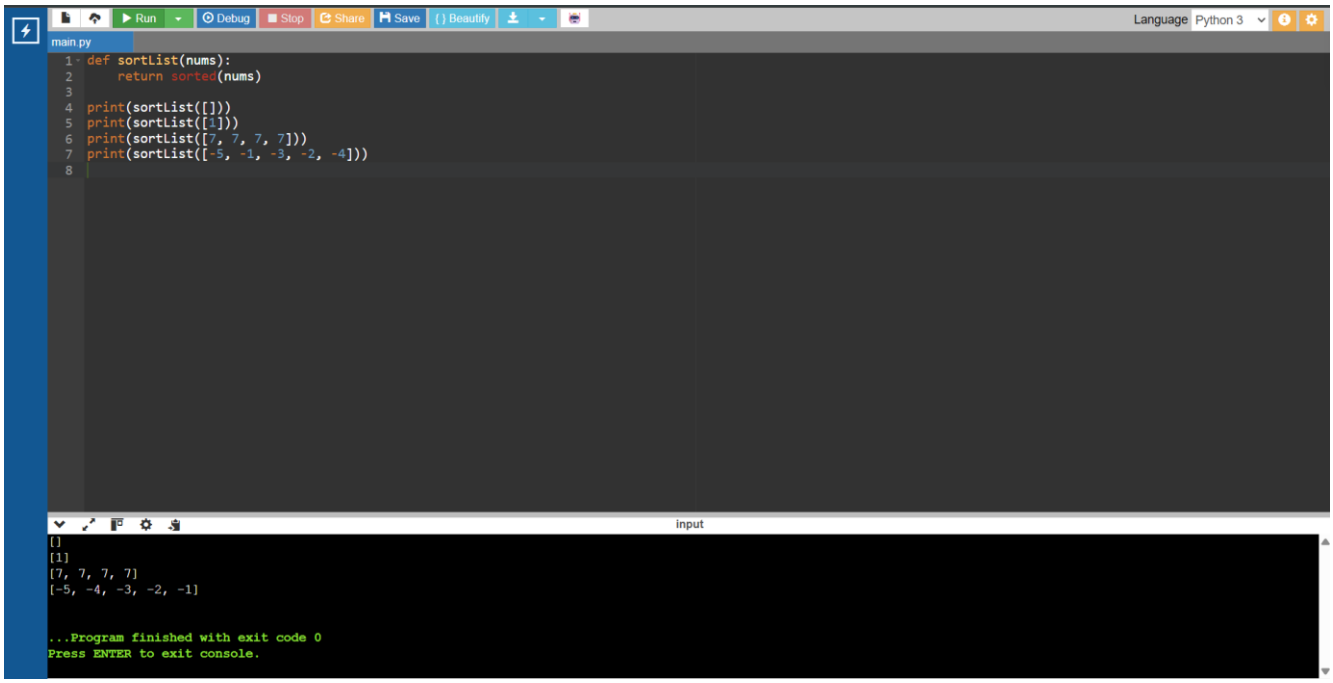
17. Sorting Lists with Different Cases

CODE:

```
def sortList(nums):
    return sorted(nums)

print(sortList([]))
print(sortList([1]))
print(sortList([7, 7, 7, 7]))
print(sortList([-5, -1, -3, -2, -4]))
```

OUTPUT:



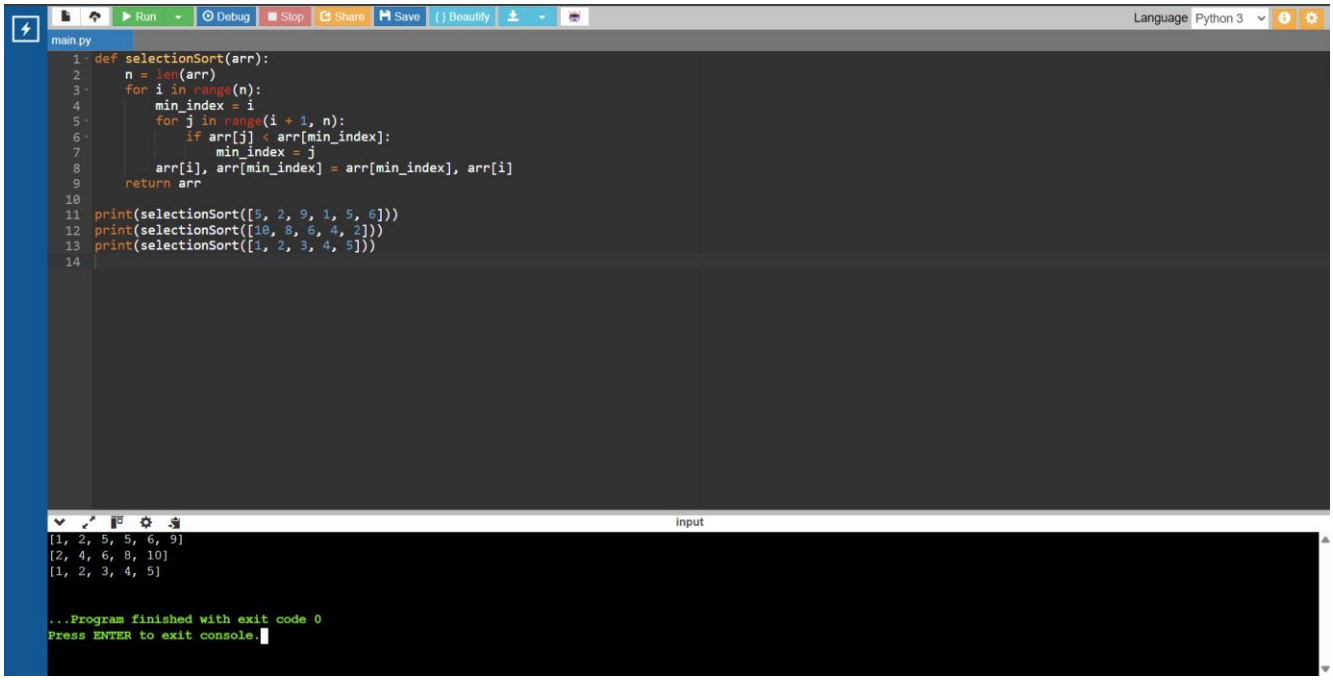
18. Selection Sort

CODE:

```
def selectionSort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

print(selectionSort([5, 2, 9, 1, 5, 6]))
print(selectionSort([10, 8, 6, 4, 2]))
print(selectionSort([1, 2, 3, 4, 5]))
```

OUTPUT:



19. Optimized Bubble Sort

CODE:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

print(bubble_sort([64, 25, 12, 22, 11]))
print(bubble_sort([29, 10, 14, 37, 13]))
print(bubble_sort([3, 5, 2, 1, 4]))
print(bubble_sort([1, 2, 3, 4, 5]))
print(bubble_sort([5, 4, 3, 2, 1]))
```

OUTPUT:



\

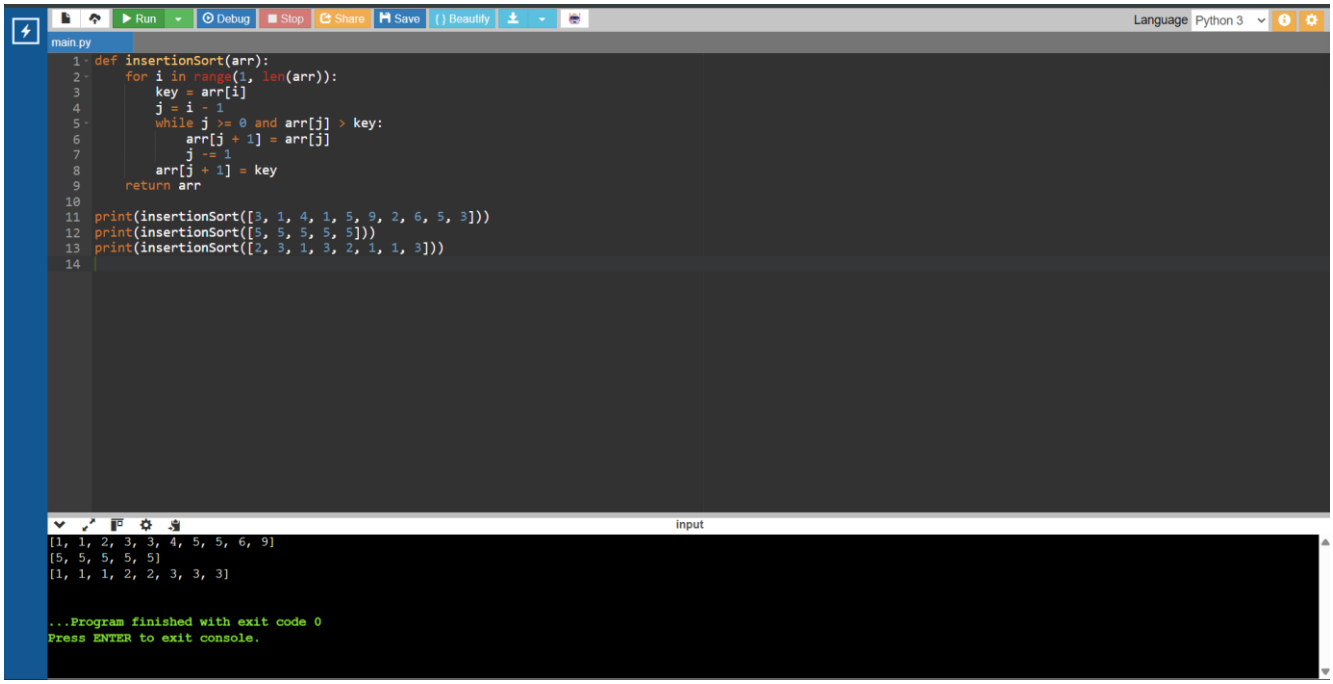
20. Insertion Sort with Duplicate Elements

CODE:

```
def insertionSort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertionSort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertionSort([5, 5, 5, 5, 5]))
print(insertionSort([2, 3, 1, 3, 2, 1, 1, 3]))
```

OUTPUT:



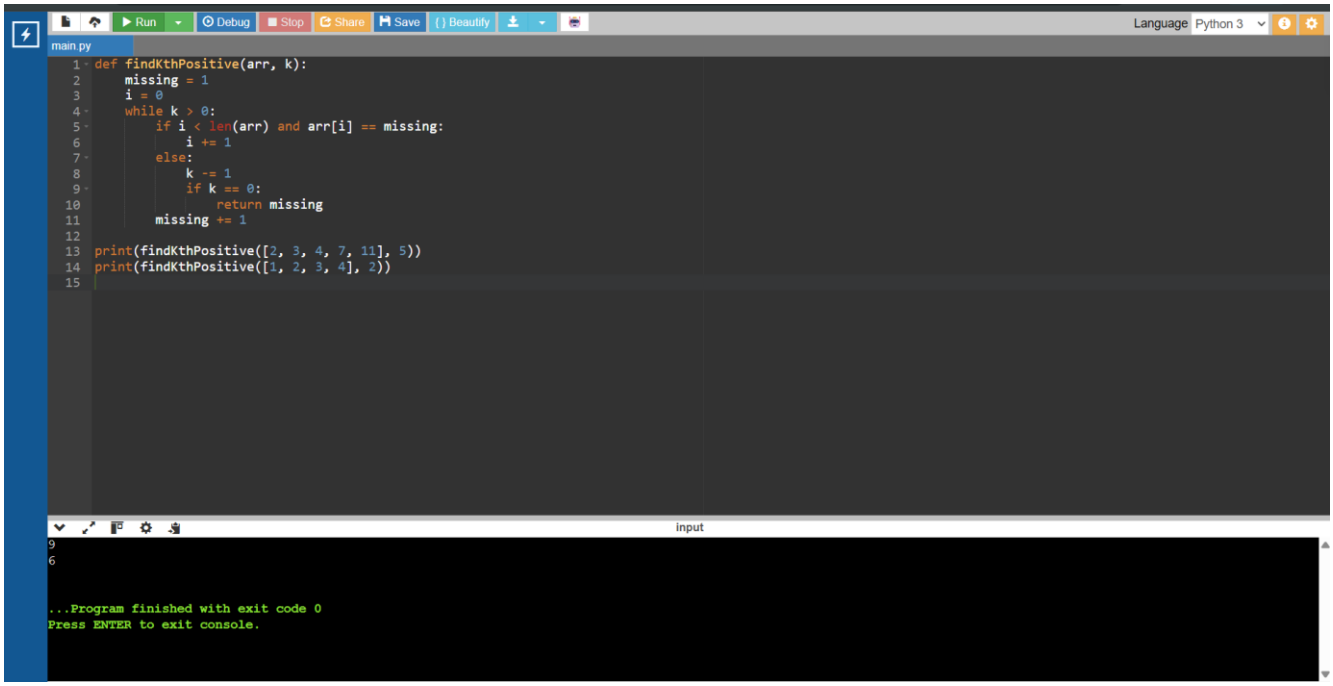
21. Kth Missing Positive Number

CODE:

```
def findKthPositive(arr, k):
    missing = 1
    i = 0
    while k > 0:
        if i < len(arr) and arr[i] == missing:
            i += 1
        else:
            k -= 1
            if k == 0:
                return missing
        missing += 1

print(findKthPositive([2, 3, 4, 7, 11], 5))
print(findKthPositive([1, 2, 3, 4], 2))
```

OUTPUT:



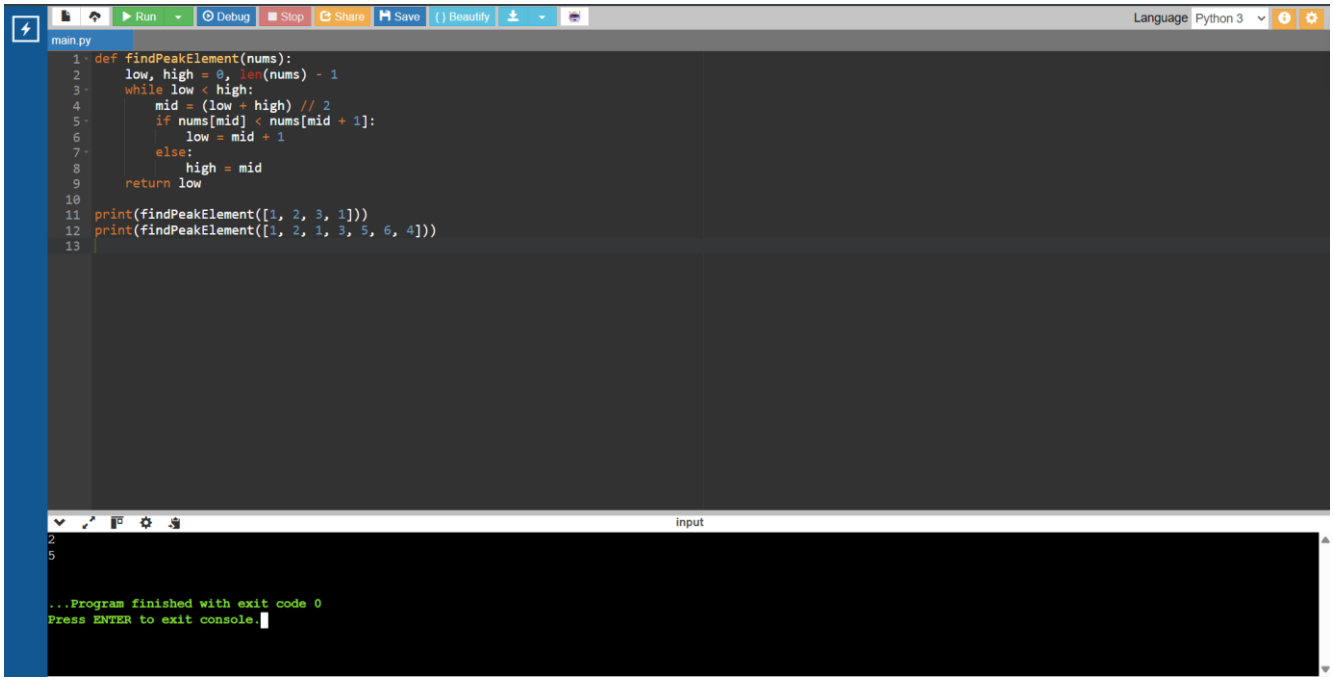
22. Find Peak Element Using Binary Search

CODE:

```
def findPeakElement(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] < nums[mid + 1]:
            low = mid + 1
        else:
            high = mid
    return low

print(findPeakElement([1, 2, 3, 1]))
print(findPeakElement([1, 2, 1, 3, 5, 6, 4]))
```

OUTPUT:



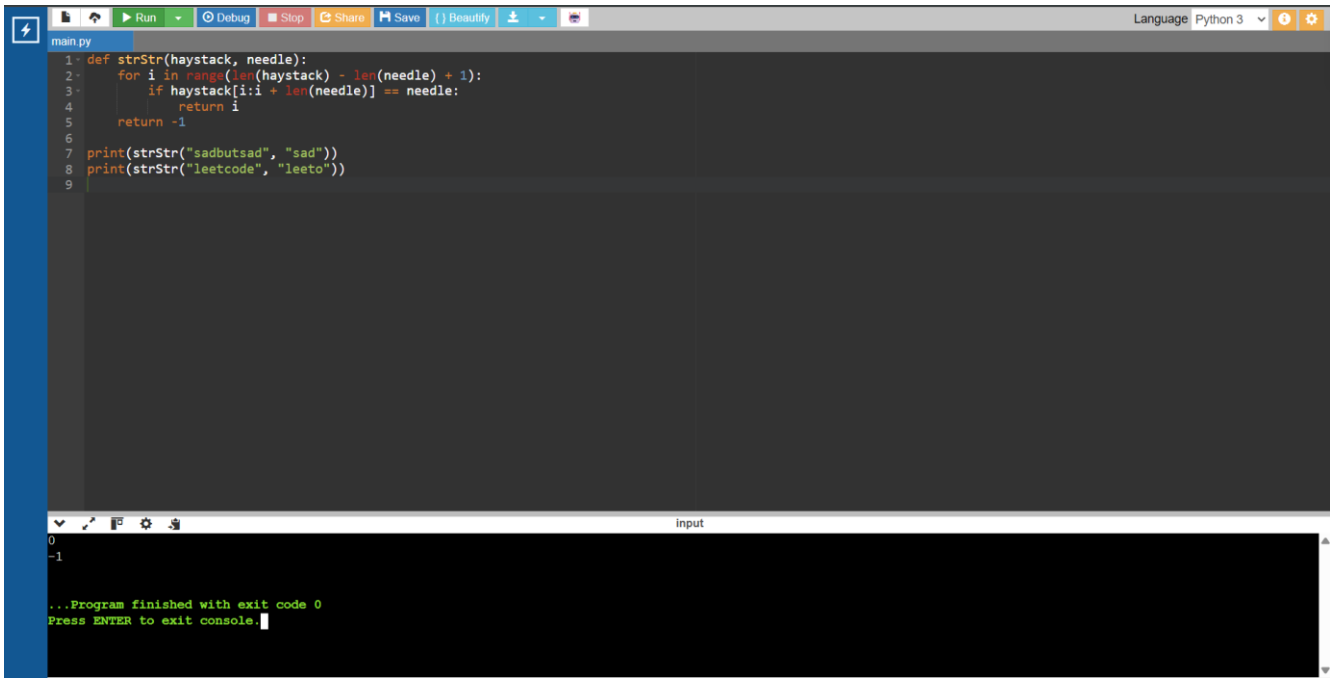
23. First Occurrence of a Substring

CODE:

```
def strStr(haystack, needle):
    for i in range(len(haystack) - len(needle) + 1):
        if haystack[i:i + len(needle)] == needle:
            return i
    return -1

print(strStr("sadbutsad", "sad"))
print(strStr("leetcode", "leeto"))
```

OUTPUT:



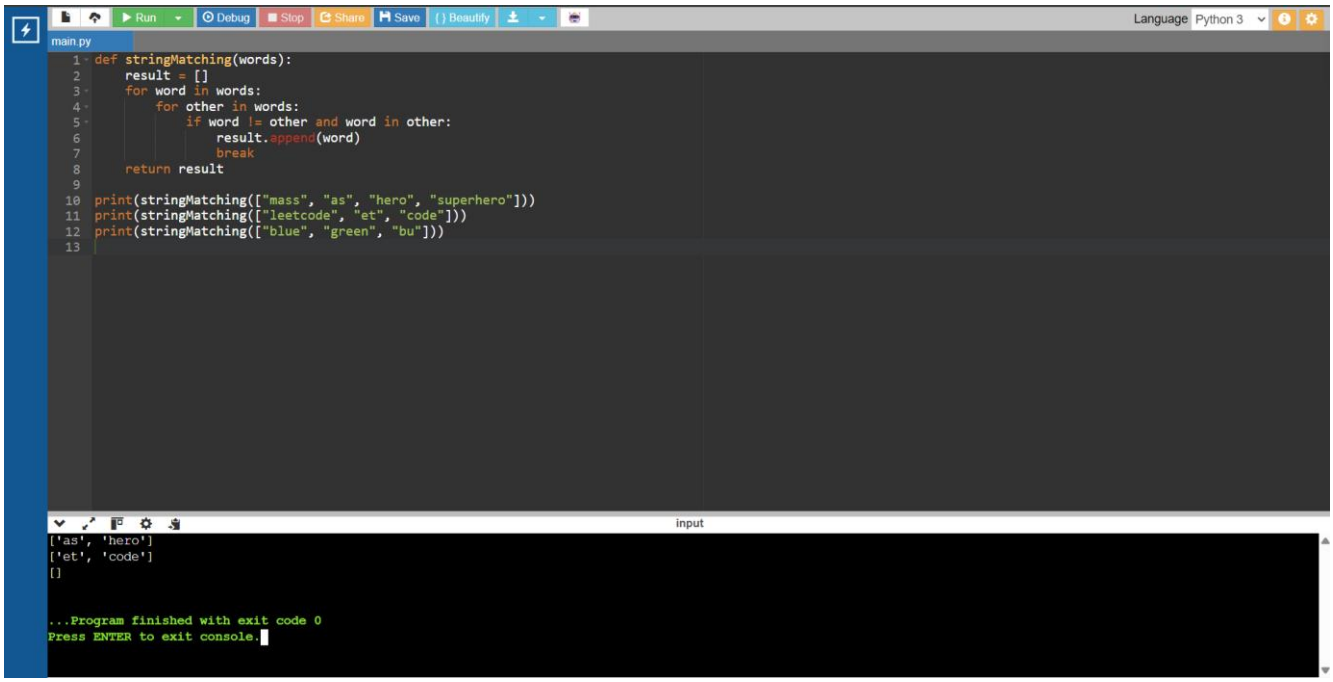
24. Find Strings That Are Substrings of Other Words

CODE:

```
def stringMatching(words):
    result = []
    for word in words:
        for other in words:
            if word != other and word in other:
                result.append(word)
                break
    return result

print(stringMatching(["mass", "as", "hero", "superhero"]))
print(stringMatching(["leetcode", "et", "code"]))
print(stringMatching(["blue", "green", "bu"]))
```

OUTPUT:



25. Closest Pair of Points (Brute Force)

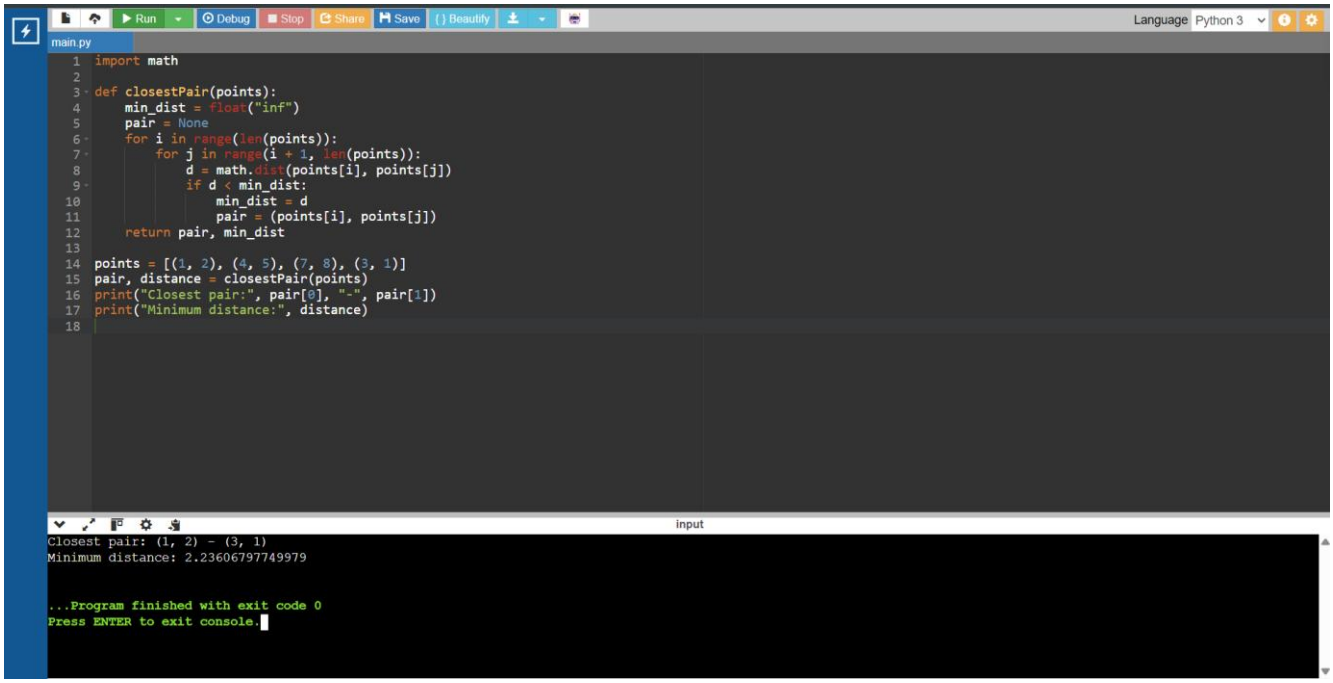
CODE:

```
import math

def closestPair(points):
    min_dist = float("inf")
    pair = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.dist(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, distance = closestPair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

OUTPUT:



26. Brute Force Convex Hull for Given Points

CODE:

```
def convexHull(points):
    hull = set()
    n = len(points)

    for i in range(n):
        for j in range(i + 1, n):
            left = right = False

            for k in range(n):
                if k == i or k == j:
                    continue
                cross = ((points[j][0] - points[i][0]) *
                         (points[k][1] - points[i][1]) -
                         (points[j][1] - points[i][1]) *
                         (points[k][0] - points[i][0]))
                if cross > 0:
                    left = True
                elif cross < 0:
                    right = True

            if not (left and right):
                hull.add(points[i])
                hull.add(points[j])

    return sorted(hull)

points = [(10, 0), (11, 5), (5, 3), (9, 3.5),
          (15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)]
print("Convex Hull:", convexHull(points))
```

OUTPUT:



27. Convex Hull of 2D Points

CODE:

```
def convexHull(points):
    hull = set()
    n = len(points)

    for i in range(n):
        for j in range(i + 1, n):
            left = right = False

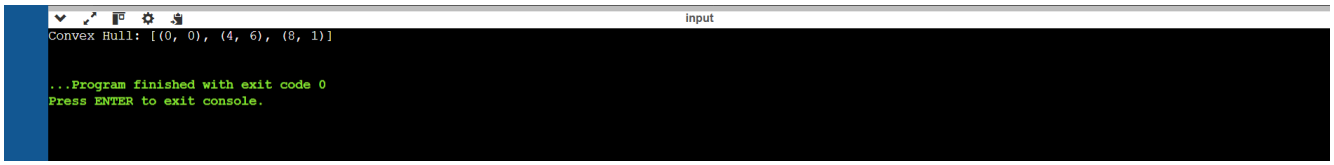
            for k in range(n):
                if k == i or k == j:
                    continue
                cross = ((points[j][0] - points[i][0]) *
                         (points[k][1] - points[i][1]) -
                         (points[j][1] - points[i][1]) *
                         (points[k][0] - points[i][0]))
                if cross > 0:
                    left = True
                elif cross < 0:
                    right = True

            if not (left and right):
                hull.add(points[i])
                hull.add(points[j])

    return sorted(hull)

points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
print("Convex Hull:", convexHull(points))
```

OUTPUT:



28. Travelling Salesperson Problem (Exhaustive Search)

CODE:

```
from itertools import permutations
from math import dist

def tsp(cities):
    start = cities[0]
    best_path = None
    best_distance = float("inf")

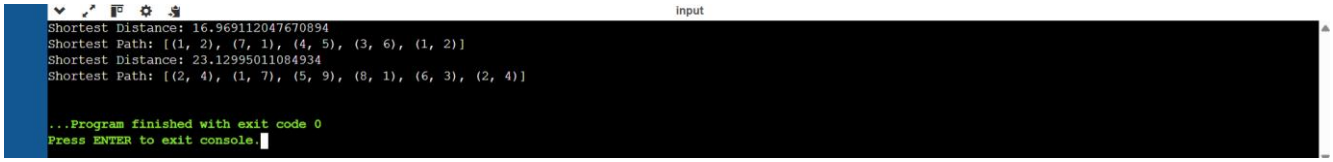
    for order in permutations(cities[1:]):
        path = [start] + list(order) + [start]
        total = sum(dist(path[i], path[i + 1]) for i in range(len(path) - 1))
        if total < best_distance:
            best_distance = total
            best_path = path

    return best_distance, best_path

cities1 = [(1, 2), (4, 5), (7, 1), (3, 6)]
cities2 = [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]

for cities in [cities1, cities2]:
    distance, path = tsp(cities)
    print("Shortest Distance:", distance)
    print("Shortest Path:", path)
```

OUTPUT:



29. Assignment Problem (Exhaustive Search)

CODE:

```
from itertools import permutations

def assignment_problem(cost_matrix):
    n = len(cost_matrix)
    best_cost = float("inf")
    best_assignment = None

    for tasks in permutations(range(n)):
        total = sum(cost_matrix[i][tasks[i]] for i in range(n))
        if total < best_cost:
            best_cost = total
            best_assignment = tasks

    pairs = [(i + 1, best_assignment[i] + 1) for i in range(n)]
    return pairs, best_cost

cost1 = [[3, 10, 7], [8, 5, 12], [4, 6, 9]]
cost2 = [[15, 9, 4], [8, 7, 18], [6, 12, 11]]

print("Optimal Assignment and Total Cost:", assignment_problem(cost1))
print("Optimal Assignment and Total Cost:", assignment_problem(cost2))
```

OUTPUT:



30. 0-1 Knapsack Problem (Exhaustive Search)

CODE:

```
from itertools import combinations

def knapsack(weights, values, capacity):
    n = len(weights)
    best_value = 0
    best_items = []

    for r in range(n + 1):
        for items in combinations(range(n), r):
            weight = sum(weights[i] for i in items)
            value = sum(values[i] for i in items)
            if weight <= capacity and value > best_value:
                best_value = value
                best_items = list(items)

    return best_items, best_value

print(knapsack([2, 3, 1], [4, 5, 3], 4))
print(knapsack([1, 2, 3, 4], [2, 4, 6, 3], 6))
```

OUTPUT:

